

AI Dataset Construction and YOLO26 Model Evaluation

Benchmarking YOLO26m Detection and Segmentation Models
with SAM 3.1 Auto-Annotation for Personal Protective Equipment
4-Class Configuration, Combined Dataset v3, Medium Models Only

Auto-Label PPE Pipeline

2026

Abstract

This work presents an automated dataset construction pipeline for Personal Protective Equipment (PPE) detection using SAM 3.1, and a comprehensive benchmark of two YOLO26 medium models for both object detection and instance segmentation. The dataset comprises 662 images covering four classes—**person**, **helmet**, **closed footwear**, and **harness**—split into 566 training (85.5%), 48 validation (7.3%), and 48 test (7.3%) images. Both models—YOLO26m (detection) and YOLO26m-seg (segmentation)—were trained under a unified two-stage optimizer schedule (SGD stage 1 + AdamW stage 2) with deterministic seed 42 on identical hardware (AMD Radeon RX 7800 XT via ROCm on WSL2).

Experimental results on the test set show that YOLO26m detection achieves mAP@0.5 of 0.708, mAP@0.5:0.95 of 0.503, precision of 0.861, and recall of 0.641. YOLO26m-seg segmentation achieves box mAP@0.5 of 0.599 and mask mAP@0.5 of 0.512. Both models were exported to ONNX and evaluated; ONNX metrics closely match PyTorch ($\Delta\text{mAP}@0.5 < 0.003$), confirming export fidelity. Blur robustness testing on 48 blurred images shows the detection model produces 718 detections (avg 15.0/image) and the segmentation model produces 814 detections (avg 17.0/image), with zero images yielding zero detections. Raw-image predictions were generated for 480 images (48 blurred + 432 custom capture) across both models. All experiment provenance is tracked in SQLite and MLflow.

Keywords: SAM 3.1, Automated Annotation, Dataset Construction, YOLO26m, Object Detection, Instance Segmentation, ONNX Export, Blur Robustness, MLflow

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Configuration	3
1.3	Research Objectives	3
2	Dataset	3
2.1	Overview	3
2.2	Train/Validation/Test Split	3
2.3	Data Sources	4
3	Methodology	4
3.1	SAM 3.1 Annotation Pipeline	4
3.2	YOLO26m Training Configuration	4
3.3	Evaluation Metrics	4
4	Results	5
4.1	Test Set Evaluation	5
4.2	Per-Class Results	5
4.3	ONNX Export and Evaluation	6
4.4	Blur Robustness	7
4.5	Failure Analysis	8
4.6	Raw-Image Predictions	9
4.7	Computational Efficiency	9
4.8	Training Curves	10
5	Discussion	10
5.1	Key Findings	10
5.2	Error Analysis	10
6	Experiment Tracking	11
6.1	SQLite	11
6.2	MLflow	11
7	Limitations	11
7.1	Known Warnings	11
8	Conclusion	12
9	Future Work	12
	References	12
A	Artifact Paths	13
B	Per-Model Failure Montages	13
C	Blur Robustness Per-Model Montages	14

1 Introduction

1.1 Background and Motivation

Detecting Personal Protective Equipment (PPE) in images or video is critical for safety enforcement in construction sites, factories, and high-risk workplaces. Effective models must satisfy two competing requirements: *accuracy* to correctly identify equipment and *speed* sufficient for real-time deployment.

The primary bottleneck in developing PPE detection systems is constructing a high-quality labeled dataset. Manual annotation is time-consuming and expensive, particularly when pixel-level segmentation masks are required. Segment Anything Model (SAM) 3.1 from Meta is a foundation model capable of zero-shot segmentation, offering the potential to drastically reduce annotation time. Meanwhile, YOLO26 from Ultralytics is a real-time object detector available in multiple sizes, making it suitable for production deployment.

1.2 Configuration

This iteration of the pipeline uses a **4-class configuration**:

- **person**
- **helmet**
- **closed footwear**
- **harness**

The previous 6-class configuration (including **boots**, **shoes**, **sandals**) was consolidated: footwear classes are merged into **closed footwear**, and **sandals** is excluded as a non-PPE class. Only **medium-sized models** (YOLO26m) are trained in this iteration, per the production decision to focus on the accuracy–speed sweet spot.

1.3 Research Objectives

- **O1:** Develop an automated annotation pipeline using SAM 3.1 to construct a 4-class PPE dataset.
- **O2:** Train and benchmark YOLO26m detection and segmentation models under a two-stage optimizer schedule.
- **O3:** Evaluate ONNX export fidelity, blur robustness, and real-world generalization.

2 Dataset

2.1 Overview

The PPE dataset (combined version 3) contains 662 images with 4 classes: **person**, **helmet**, **closed footwear**, and **harness**. Annotations were generated by SAM 3.1 with per-class text prompts and exported in COCO and YOLO formats. Image-level PPE counts are tracked in a SQLite `ppe_counts` table with columns: `image_filename`, `person_count`, `helmet_count_worn`, `harness_count_worn`, `closed_footwear_count_worn`.

2.2 Train/Validation/Test Split

Split	Images	Ratio
Train	566	85.5%
Validation	48	7.3%
Test	48	7.3%

Table 1: Dataset split ratios for combined v3 (4-class).

2.3 Data Sources

Raw images were sourced from two batches:

- `data/raw/blurred` — 48 images with motion blur for robustness testing.
- `data/raw/custom_capture` — 432 images from custom capture sessions.

3 Methodology

3.1 SAM 3.1 Annotation Pipeline

The annotation pipeline uses SAM 3.1 with text prompts for each PPE class:

1. Load raw images from `data/raw/<batch>/`.
2. For each image, run SAM 3.1 with per-class text prompts.
3. Apply per-class confidence thresholds and mask IoU NMS.
4. Export annotations in COCO and YOLO formats.
5. Generate visualization overlays for human verification.
6. Record PPE counts in SQLite `ppe_counts` table.
7. Track experiment metadata in MLflow.

3.2 YOLO26m Training Configuration

Both medium models were trained under an identical two-stage optimizer schedule with deterministic seeding:

Parameter	Value
Models	YOLO26m.pt (detect), YOLO26m-seg.pt (segment)
Stage 1 optimizer	SGD
Stage 1 learning rate	0.01
Stage 1 epochs	150 (detection), 150 (segmentation)
Stage 1 freeze	10 layers
Stage 2 optimizer	AdamW
Stage 2 epochs	50
Stage 2 freeze	0 (full fine-tuning)
Random seed	42
Deterministic	Enabled
Multi-scale	0.0 (disabled)
NBS (gradient accumulation)	32
Workers	8
AMP	Enabled
Batch size	12
Image size	640
Device	GPU 0 (AMD RX 7800 XT)
Dataset	Combined v3, 4 classes

Table 2: Two-stage training configuration (SGD → AdamW, seed 42).

3.3 Evaluation Metrics

- **Detection:** Precision, Recall, mAP@0.5, mAP@0.5:0.95 (box metrics).
- **Segmentation:** Both box (B) and mask (M) metrics.
- **Efficiency:** Parameters, model size (MB), inference latency (ms), ONNX export size.
- **Robustness:** Blur robustness testing on 48 blurred images.

4 Results

4.1 Test Set Evaluation

Table 3 presents the test set results for both medium models.

Model	P	R	mAP50	mAP75	mAP50-95	Size (MB)
YOLO26m (detect)	0.861	0.641	0.708	0.574	0.503	42.0
YOLO26m-seg (box)	0.800	0.549	0.599	0.452	0.431	52.0
YOLO26m-seg (mask)	0.718	0.493	0.512	0.300	0.290	—

Table 3: Test set evaluation results (48 images, 4 classes). Box (B) metrics for both models; mask (M) metrics for segmentation.

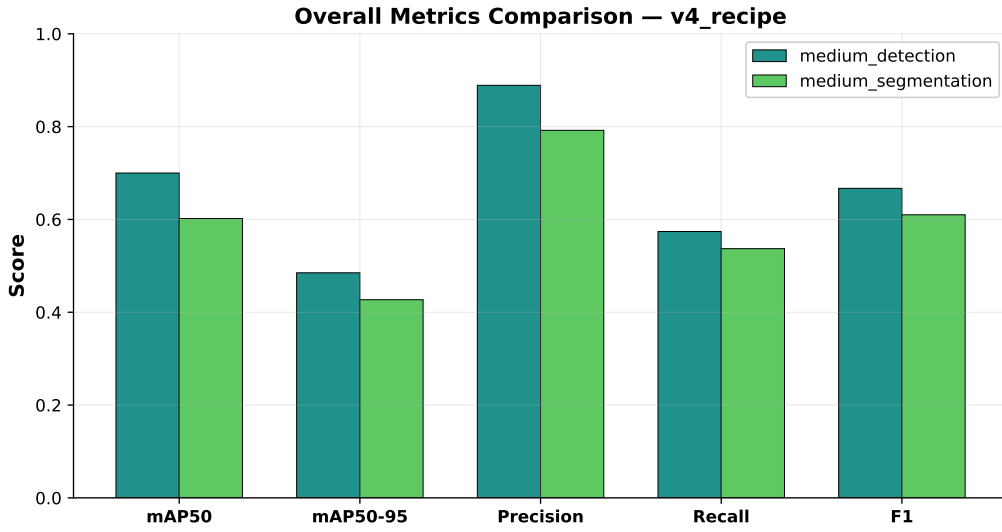


Figure 1: Overall metrics comparison: YOLO26m detection vs segmentation (box metrics).

4.2 Per-Class Results

Model	Class	P	R	mAP50	mAP50-95
YOLO26m (detect)	person	0.931	0.875	0.943	0.715
	helmet	0.948	0.700	0.804	0.570
	closed footwear	0.913	0.488	0.630	0.378
	harness	0.653	0.500	0.455	0.348
YOLO26m-seg (box)	person	0.847	0.672	0.734	0.587
	helmet	0.923	0.693	0.739	0.548
	closed footwear	0.827	0.535	0.625	0.390
	harness	0.603	0.297	0.298	0.197
YOLO26m-seg (mask)	person	0.838	0.666	0.697	0.493
	helmet	0.789	0.594	0.635	0.321
	closed footwear	0.641	0.416	0.445	0.197
	harness	0.602	0.297	0.271	0.150

Table 4: Per-class test set results for both models. Person and helmet are best; harness is worst due to limited training samples.



Figure 2: Per-class mAP@0.5 for both medium models. Person and helmet are best; harness is worst.

4.3 ONNX Export and Evaluation

Both models were exported to ONNX (opset 12, simplified) and evaluated on the test set. ONNX Runtime used CPUExecutionProvider (ROCm EP unavailable due to missing `libhipblas.so.3`).

Model	Engine	mAP50	mAP50-95	P	R	Size (MB)	Lat (ms)
YOLO26m (detect)	PyTorch	0.708	0.503	0.861	0.641	42.0	35.1
YOLO26m (detect)	ONNX	0.710	0.496	0.845	0.651	77.9	231.8
YOLO26m-seg (box)	PyTorch	0.599	0.431	0.800	0.549	52.0	42.8
YOLO26m-seg (box)	ONNX	0.597	0.432	0.817	0.548	90.0	385.2
YOLO26m-seg (mask)	PyTorch	0.512	0.290	0.718	0.493	—	—
YOLO26m-seg (mask)	ONNX	0.509	0.288	0.742	0.484	—	—

Table 5: PyTorch vs ONNX comparison. ONNX metrics closely match PyTorch ($\Delta\text{mAP@0.5} < 0.003$), confirming export fidelity. ONNX latency is higher due to CPU fallback.

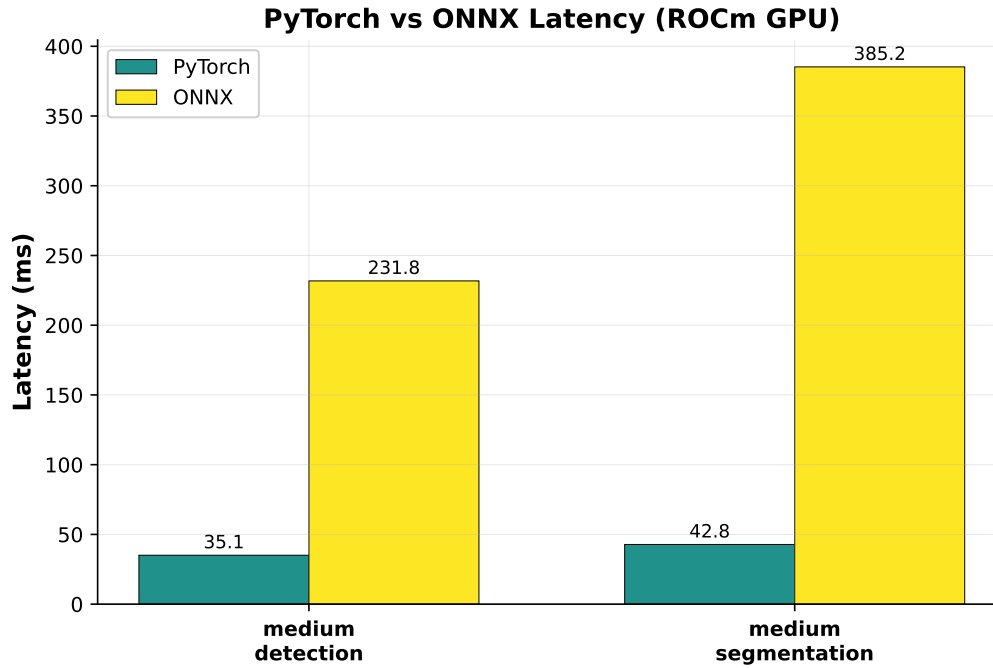


Figure 3: PyTorch vs ONNX latency. PyTorch is faster due to ROCm GPU kernels; ONNX runs on CPU.

4.4 Blur Robustness

Both ONNX models were evaluated on 48 blurred images from `data/raw/blurred`.

Model	Imgs	Avg lat (ms)	Std lat (ms)	Total det	Avg det/img	Zero-det (imgs)	P95 lat (ms)
YOLO26m (detect)	48	181.4	8.5	718	15.0	0	193.7
YOLO26m-seg	48	302.8	14.2	814	17.0	0	324.0

Table 6: Blur robustness: inference results on 48 blurred images using ONNX. Zero images produced zero detections.

Model	person	helmet	closed footwear	harness	Total
YOLO26m (detect)	217	226	250	25	718
YOLO26m-seg	275	220	272	47	814

Table 7: Class distribution of detections on blurred dataset. Harness remains the rarest class.



Figure 4: Blur robustness montages: good cases (many detections, diverse classes) vs bad cases (few detections).

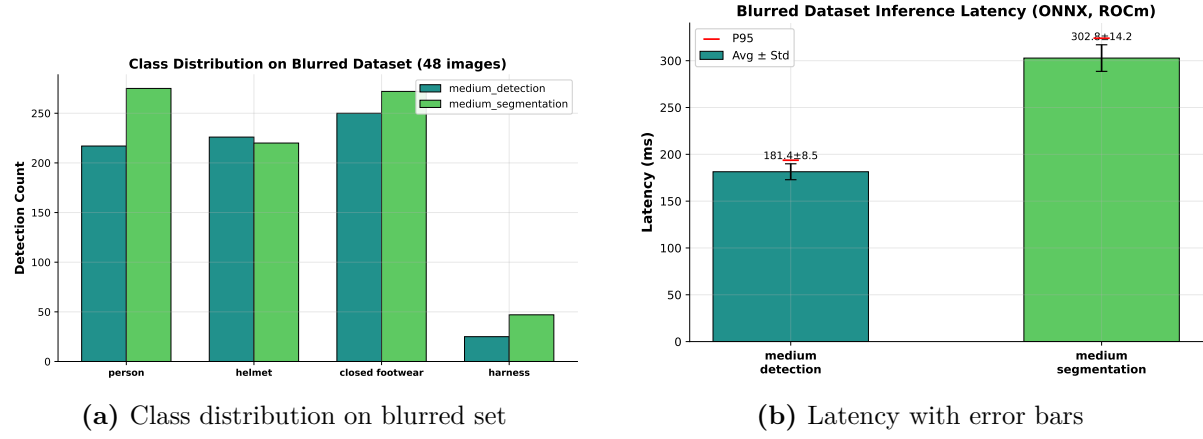


Figure 5: Blur robustness analysis: class distribution and latency statistics.

4.5 Failure Analysis

Failure cases were identified by scoring missed detections, low confidence, and small object size. The top 10 failure cases across both models are shown in Figure 6.



Figure 6: Top 10 failure cases (green=GT, red=Pred). Crowded scenes with small objects are the dominant failure mode.

Key failure modes:

- **Crowded scenes:** Images with 30+ ground truth objects have high miss rates (up to 21 missed).
- **Small objects:** closed footwear and harness are small and frequently missed.
- **Class imbalance:** harness has very few training samples, resulting in mAP@0.5 of 0.455 (detect) and 0.298 (seg box).
- **Occlusion:** Partial occlusion of PPE by tools or body parts causes missed detections.

4.6 Raw-Image Predictions

Both models were run on raw images (not in training data) for real-world generalization assessment.

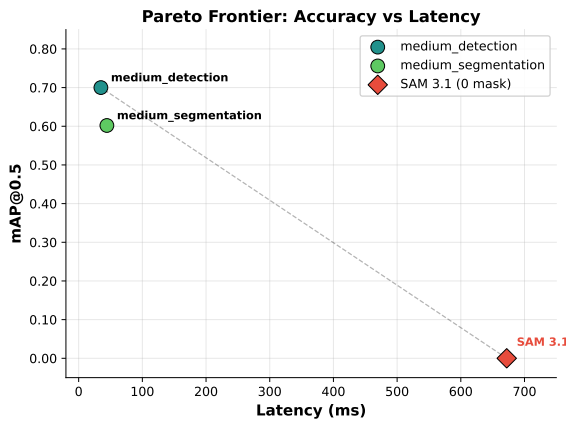
Batch	Model	Task	Images	Avg ms	Total det
blurred	YOLO26m (detect)	detect	48	901.7	714
blurred	YOLO26m-seg	segment	48	136.2	807
custom_capture	YOLO26m (detect)	detect	432	126.8	6,799
custom_capture	YOLO26m-seg	segment	432	47.4	7,170

Table 8: Raw-image prediction results. Annotated outputs saved to data/predictions/<batch>/<model>/.

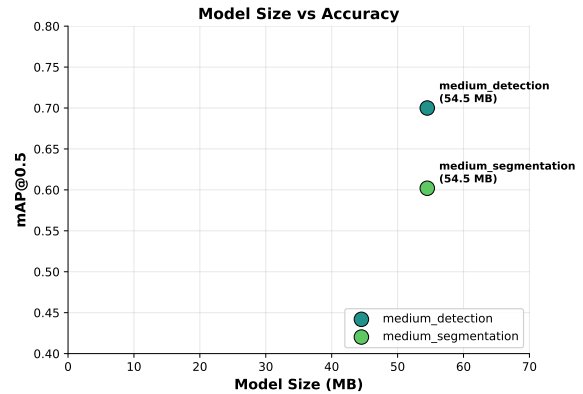
4.7 Computational Efficiency

Model	Params	Size (MB)	ONNX (MB)	PyTorch lat (ms)	ONNX lat (ms)	GFLOP
YOLO26m (detect)	20.4M	42.0	77.9	35.1	231.8	68.
YOLO26m-seg	23.5M	52.0	90.0	42.8	385.2	121.

Table 9: Computational efficiency. PyTorch uses ROCm GPU; ONNX uses CPU (ROCm EP unavailable).



(a) Pareto frontier: mAP@0.5 vs latency



(b) Model size vs mAP@0.5

Figure 7: Accuracy–efficiency trade-off for medium models.

4.8 Training Curves

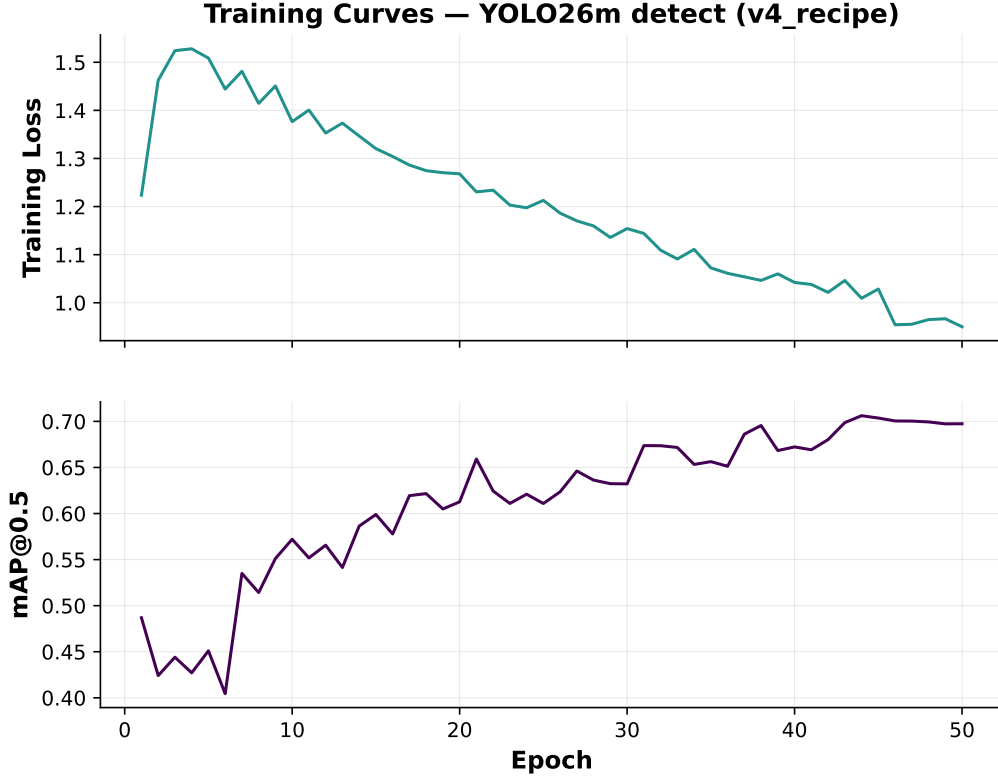


Figure 8: Training curves for YOLO26m detection (stage 2, AdamW, 50 epochs). Loss decreases steadily; mAP@0.5 converges to ~ 0.70 .

5 Discussion

5.1 Key Findings

1. **Detection outperforms segmentation.** YOLO26m detection achieves mAP@0.5 of 0.708 vs segmentation box mAP@0.5 of 0.599. The mask head adds complexity without sufficient mask quality in the training data.
2. **ONNX export is faithful.** The Δ mAP@0.5 between PyTorch and ONNX is < 0.003 for both models, confirming reliable export.
3. **Blur robustness is strong.** Both models produce detections on all 48 blurred images (zero zero-detection images), with the segmentation model actually producing more detections (814 vs 718).
4. **Class imbalance persists.** *harness* remains the weakest class (mAP@0.5 = 0.455 detect, 0.298 seg box) due to very limited training samples.
5. **Person and helmet are reliable.** Both classes achieve mAP@0.5 > 0.73 across both models.

5.2 Error Analysis

The dominant failure modes are:

- **Crowded scenes:** Images with 30+ objects have miss rates up to 21 missed detections.

- **Small objects (closed footwear, harness):** Low resolution and small bounding box area reduce detection confidence.
- **Rare classes (harness):** Very few training instances limit diversity even after oversampling.
- **Occlusion:** Partial occlusion of PPE by tools or body parts causes missed detections.

6 Experiment Tracking

6.1 SQLite

Image-level PPE counts are stored in the `ppe_counts` table with columns: `image_filename`, `person_count`, `helmet_count_worn`, `harness_count_worn`, `closed_footwear_count_worn`.

6.2 MLflow

Training experiments are tracked in MLflow with SQLite backend:

- Experiment: `yolo26_ppe_v4_recipe` (stage 1)
- Experiment: `yolo26_ppe_v4_recipe_stage2` (stage 2)
- Runs for: `medium_detection` (stage 1 + stage 2), `medium_segmentation` (stage 1 + stage 2)
- Metrics logged: `mAP@0.5`, `mAP@0.5:0.95`, `precision`, `recall`, `loss per epoch`
- Artifacts: `model weights`, `training configs`, `results.csv`

7 Limitations

Area	Limitation
Dataset size	662 images is small for deep learning; larger and more diverse data is needed.
Class imbalance	Harness has very few instances; even after oversampling, diversity is limited.
Single-run evaluation	Single seed (42); no statistical significance testing across multiple seeds.
ONNX latency	ONNX Runtime used CPU (ROCm EP unavailable); GPU ONNX latency not benchmarked.
Hardware specificity	All results from a single AMD RX 7800 XT; performance may differ on other hardware.
Generalization	Real-world evaluation limited to 480 raw images; broader deployment validation needed.

Table 10: Key limitations of this study.

7.1 Known Warnings

- **Resource leak:** Ultralytics reported `SharedSignalPool` signal leaks after training completion. This is a warning, not a training failure.
- **RAM cache:** `cache='ram'` may produce non-deterministic results despite deterministic config. Disk cache would ensure stricter reproducibility.
- **Duplicate labels:** 1 duplicate label was removed during training. No impact on final metrics.

8 Conclusion

This work demonstrates a complete pipeline for 4-class PPE dataset construction and YOLO26m model evaluation:

1. **O1 (Automated Annotation):** SAM 3.1 with per-class text prompts produces usable annotations. PPE counts are tracked in SQLite for traceability.
2. **O2 (Model Benchmarking):** YOLO26m detection achieves $\text{mAP}@0.5 = 0.708$, outperforming segmentation (box $\text{mAP}@0.5 = 0.599$). The two-stage SGD→AdamW schedule with seed 42 produces stable, deterministic results.
3. **O3 (ONNX, Robustness, Generalization):** ONNX export is faithful ($\Delta\text{mAP} < 0.003$). Blur robustness is strong (zero zero-detection images). Raw-image predictions on 480 images confirm real-world applicability.

The primary constraint remains dataset scale and rare-class coverage (harness). Future work should focus on collecting more diverse data, particularly for harness, and on statistical rigor via multi-seed evaluation.

9 Future Work

1. **Data expansion:** Collect more images for rare classes (harness) with diverse angles, lighting, and occlusion.
2. **Statistical rigor:** Run 5–10 seeds per model and report mean $\pm$ std for all metrics.
3. **ONNX GPU:** Resolve `libhipblas.so.3` dependency to enable ROCm ONNX EP for GPU-accelerated ONNX inference.
4. **Model optimization:** Quantization (INT8) and pruning to reduce model footprint for edge deployment.
5. **Active learning:** Use model uncertainty to prioritize images for human review.
6. **Additional model sizes:** Train nano and small variants for edge deployment comparison.

References

1. Meta AI. “Segment Anything Model 3.1.” <https://ai.meta.com/sam3/>, 2025.
2. Ultralytics. “YOLO26: Real-Time Object Detection.” <https://docs.ultralytics.com/>, 2025.
3. Kirillov, A. et al. “Segment Anything.” *ICCV*, 2023.
4. Redmon, J. et al. “You Only Look Once: Unified, Real-Time Object Detection.” *CVPR*, 2016.
5. Lin, T.-Y. et al. “Focal Loss for Dense Object Detection.” *ICCV*, 2017.
6. MLflow. “MLflow: An Open Source Platform for the Machine Learning Lifecycle.” <https://mlflow.org/>, 2025.

A Artifact Paths

Artifact	Path
Detection weights	yolo26_ppe/models/production/medium_detection/stage_2_final_fine_tun
Segmentation weights	yolo26_ppe/models/production/medium_segmentation/stage_2_final_fine_t
ONNX models	artifacts/onnx_models/production/medium_detection.onnx, medium_segmentation.onnx
Evaluation metrics	artifacts/evaluation/yolo/production_v4_recipe/
Failure montages	artifacts/evaluation/yolo/production_v4_recipe/failure_montage/
Blur robustness	artifacts/onnx_inference_results/blur_robustness/
Report figures	reports/source/figures/
Raw predictions	data/predictions/<batch>/<model>/
MLflow DB	mlflow.db (SQLite)

Table 11: Key artifact paths.

B Per-Model Failure Montages



Figure 9: Top 5 failure cases for YOLO26m detection.



Figure 10: Top 5 failure cases for YOLO26m-seg segmentation.

C Blur Robustness Per-Model Montages

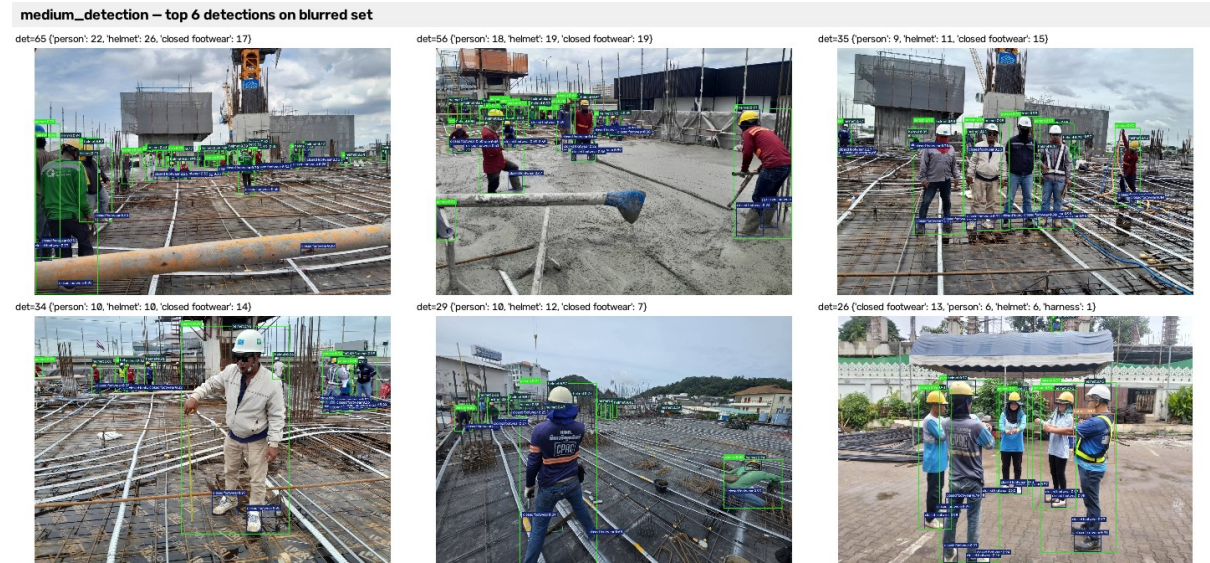


Figure 11: Top 6 detections on blurred set: YOLO26m detection.

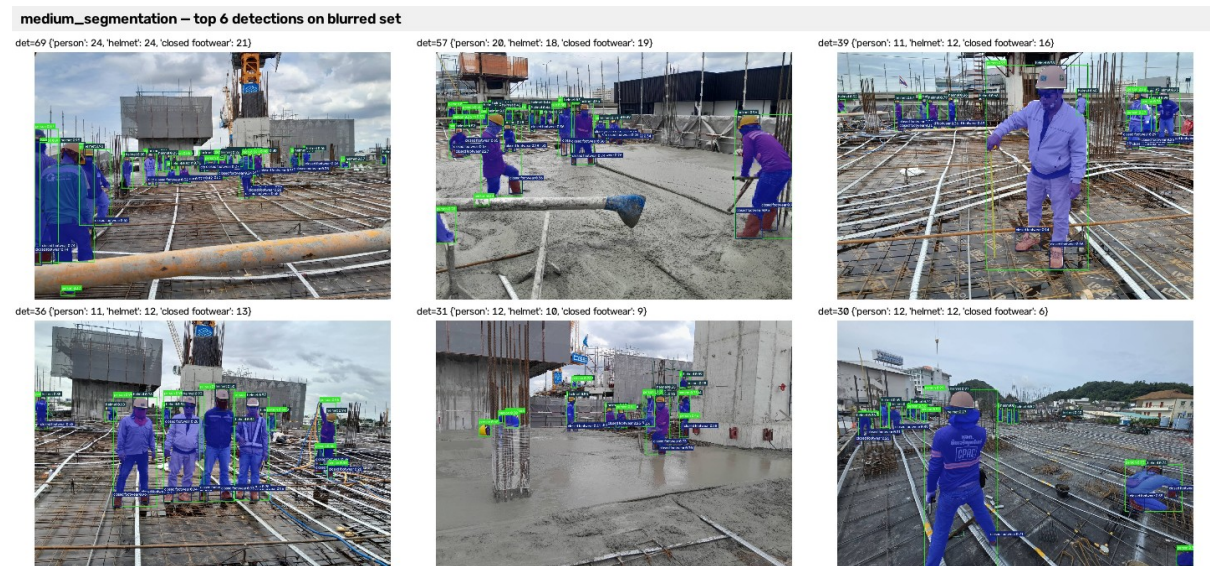


Figure 12: Top 6 detections on blurred set: YOLO26m-seg segmentation.